

Chat Transcript

Prompts and responses before the PDF request

Task 1: Fix the power pellet bug

User prompt

Eating a power pellet is supposed to send every ghost into frightened mode for a few seconds, reversing their direction and making them vulnerable. In the current build, eating a power pellet does nothing beyond the usual 10 points - frightened mode never triggers. The maze layout marks power pellets with one specific character; check what character `eat()` compares a cell against, versus what the maze grid actually uses for power pellets.

Assistant update

I'll trace the pellet marker from the maze data into `eat()` and make the smallest fix so power pellets trigger frightened mode.

Assistant response

Fixed the power pellet check in `game.py`: `eat()` now checks for lowercase "o", matching the maze grid. Eating a power pellet will trigger frightened mode and reverse each non-eaten ghost.

Task 2: Implement `ghost_color(name, mode)`

User prompt

Called once per ghost per frame in `draw`, as `color = ghost_color(ghost.name, mode)` or `color`, after the default color for that ghost/mode has already been computed. `name` is one of "blinky", "pinky", "inky", "clyde"; `mode` is "normal", "frightened", or "eaten". Return an (r, g, b) override, or `None` to keep the default. Idea: give each ghost a distinct frightened-mode tint instead of the shared blue/white flash.

Assistant update

I'll implement per-ghost frightened tints while returning `None` for normal and eaten modes so their existing colors stay intact.

Assistant response

Implemented `ghost_color()` in `game.py`. Each ghost now gets a distinct frightened-mode tint; normal and eaten modes keep their existing default colors. Unknown names return `None`.

Task 3: Implement `on_pellet_eaten(score, pellets_left)`

User prompt

Called from `eat()` immediately after every pellet (regular or power) is removed from the board and the score updated. It receives the score after this pellet's points were added, and the number of pellets still remaining. Its return value is ignored. Idea: spawn a bonus fruit once `pellets_left` crosses a specific number, or flash the HUD on the final pellet.

Assistant update

I'll use the final-pellet HUD flash option. The callback has no game instance, so I'll keep a small module-level timer and let the game update and draw consume it.

Assistant response

Implemented a 1.2-second HUD flash when the final pellet is eaten. The callback starts the flash, and the game updates its timer even after entering the win state. Normal pellets leave the HUD unchanged.

Task 4: Implement `bonus_life_threshold()`

User prompt

Called every frame in `update()`. It takes no arguments and should return an integer score value, or `None` to disable bonus lives entirely. Whenever the player's score crosses a multiple of that value for the first time, one life is awarded automatically - the surrounding bookkeeping (`self.bonus_awarded`) is already implemented, so you only need to choose or compute the threshold. Idea: return a fixed value like 10000.

Assistant update

I'll enable the existing bonus-life bookkeeping with a 10,000-point threshold.

Assistant response

Implemented `bonus_life_threshold()` in `game.py` to return 10000. The existing update logic will award one extra life each time the score crosses another 10,000-point multiple.